ProcureMind: Agentic LLM-Based Invoice Authentication with Goods-Receipt Matching for First-Time Suppliers

Shrinidhi M
Presidency University
Bangalore, India

Nishanth D
Presidency University
Bangalore, India

Sinchana R T
Presidency University
Bangalore, India

Pavan K L
Presidency University
Bangalore, India

Ms. Priyanki Dutta
(Project Guide)
Presidency University
Bangalore, India

***Abstract*—Invoice fraud such as changed bank details, look-alike vendors, duplicate billing and inflated totals continues to cost organisations heavily. Most accounts-payable controls rely on a vendor master file and a purchase order, so they cannot vouch for a supplier that has never been registered. This paper presents ProcureMind, an invoice-authentication system that combines three parts. A multimodal large language model (LLM) reads each invoice into structured fields. A deterministic engine of nine rule checks compares those fields with the organisation's own records. A tool-using LLM agent then investigates whatever the rules flag. For first-time suppliers we introduce a goods-receipt matching method: invoice lines are matched to warehouse goods received notes (GRNs) by product similarity, then quantities and unit costs are compared, so the physical delivery becomes the evidence of authenticity. The final decision takes the stricter of the rule-based risk score and the agent's score, so the LLM can raise risk but never clear a failed check. On a benchmark of ten invoices covering nine behaviours, run three times end to end with a production LLM, the system flagged every anomalous invoice in all 30 analyses (recall 1.00) and extracted every key field correctly, with a mean of 12.5 s per invoice. The agent escalated a bank-detail-change attack to rejection in every trial, but it also caused all three false positives, which came on genuine invoices; on this benchmark the rule engine alone was exact. A separate benchmark of the line matcher ranked the correct item first for all 36 paraphrased descriptions and accepted 91.7% of them at its operating threshold. It made no false matches on unrelated products but 3 on 8 deliberately similar ones. The system runs serverless on Cloudflare Workers with a PostgreSQL database.**

***Index Terms*—invoice fraud detection, accounts payable, large language models, LLM agents, tool use, document understanding, three-way matching, goods received note, procurement.**

I. Introduction

Paying a supplier is one of the last points at which an organisation can stop a fraudulent transaction cheaply. Billing schemes are among the most common forms of occupational fraud, and their median loss is among the highest [1]. Business e-mail compromise is often carried out by sending a genuine-looking invoice with new bank details, and it caused reported losses of about US$2.9 billion in 2023 [2]. These attacks need no sophisticated forgery. A plausible vendor name, a believable amount and a note such as *"our account is frozen, please pay the new account"* are often enough to get past a busy accounts-payable (AP) clerk.

The usual defence is a set of internal controls [3]. The supplier must appear in an approved vendor master. The invoice must quote an open purchase order (PO). Ideally, the goods must have been received before payment, which is the classic *three-way match* of PO, goods receipt and invoice. In practice these checks are often done by hand, inconsistently and under time pressure. Their weakest point is the *first-time supplier*: a vendor that is not yet in the registry has no tax ID or bank account on file to compare against. Such invoices are therefore either rejected outright, which blocks legitimate emergency purchases, or approved on trust, which is exactly the gap that impersonators exploit.

Large language models (LLMs) with vision and tool use make it possible to automate the reading and the reasoning in this process. Used alone, however, an LLM is a poor control. Its judgments are not reproducible, it can hallucinate [4], and it can be persuaded by the persuasive text that fraudulent invoices contain. This paper describes **ProcureMind**, a system that uses LLMs where they are strong (reading documents and investigating open-ended anomalies) while keeping the payment decision anchored to deterministic checks against the organisation's own records. Our contributions are:

- a hybrid pipeline that combines LLM extraction, a nine-check rule engine and a tool-using review agent, with a *stricter-signal-wins* fusion rule under which the agent can raise risk but never override a failed check;
- a goods-receipt matching method that authenticates first-time suppliers against warehouse delivery records. It covers supplier identity resolution, line-level product matching with quantity and price corroboration, and protection against billing the same delivery twice;
- an end-to-end evaluation on a ten-invoice benchmark covering nine behaviours, with ablations of the rule-only and agent-only paths and a stand-alone benchmark of the product matcher;
- a serverless deployment on Cloudflare Workers with PostgreSQL that streams each stage of the analysis live to the reviewer.

II. Literature Review

A. Financial Fraud Detection

Statistical and machine-learning fraud detection has been surveyed extensively [5]–[7]. Most of this work targets high-volume transaction streams such as credit cards, telecommunications and insurance claims, where supervised classifiers can learn from millions of labelled examples. Procurement fraud is different: each organisation sees few invoices, fraud is rare, and labels are scarce. Forensic accountants have therefore relied on analytical tests such as Benford's law on invoice amounts and duplicate-payment detection [8], and on graph and network analysis of vendor relationships [9]. These methods operate on data that has already been keyed in and paid, rather than at the moment an invoice arrives. Their results are also statistical indicators, not explanations an AP clerk can act on.

B. Document Understanding

Extracting fields from invoices is a long-standing document-understanding problem. Chargrid [10] encoded page layout as a two-dimensional character grid. LayoutLM [11] and LayoutLMv3 [12] pre-trained transformers on text, layout and image together, and Donut [13] removed the separate OCR stage altogether. Public benchmarks such as FUNSD [14], SROIE [15] and CORD [16] measure key-information extraction from forms and receipts. These models need fine-tuning for each document schema. General multimodal LLMs [17], by contrast, can read an unseen invoice layout zero-shot and return output that conforms to a JSON schema. Extraction alone, however, says nothing about whether the invoice is genuine.

C. LLM Agents and Tool Use

Chain-of-thought prompting [18] showed that LLMs reason better when they spell out intermediate steps. ReAct [19] interleaves this reasoning with actions such as tool calls, and Toolformer [20] showed that models can learn when to call external APIs. These ideas make it practical for an agent to query a vendor registry or an invoice history while it investigates. LLMs nonetheless hallucinate [4], and their numerical judgments are poorly calibrated. In a financial control, these weaknesses argue for keeping deterministic checks in charge of the decision.

D. Research Gap

We found no prior system that (i) reads arbitrary invoices without template training, (ii) applies auditable rule checks against the organisation's own registry, POs and history, (iii) lets an agent investigate the residual risk with tools, and (iv) authenticates unregistered suppliers using physical goods-receipt evidence. ProcureMind addresses these four points together.

III. Proposed Methodology

Fig. 1 shows the pipeline. Each uploaded PDF or image passes through four stages, and every stage is streamed live to the reviewer as newline-delimited JSON events.

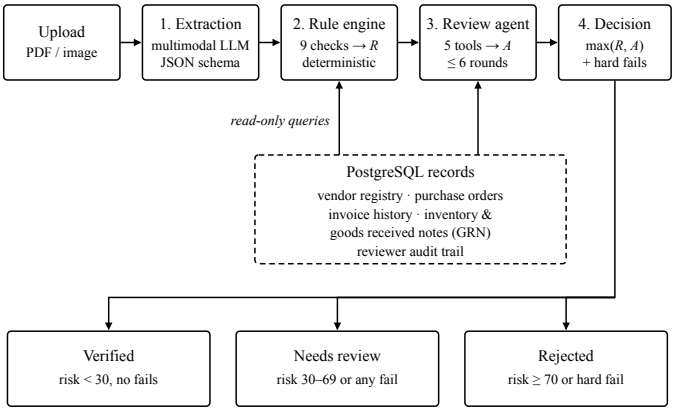


Fig. 1. ProcureMind pipeline. The rule engine and the review agent both query the organisation's records. The rule score R and the agent score A are fused by taking the stricter of the two, and a human reviewer can override any outcome.

A. Stage 1: Structured Extraction

The document is sent to a multimodal LLM through the OpenAI Responses API with *structured outputs*, which force the reply to conform to a JSON schema (a Zod schema in our code). The schema holds about twenty fields, including invoice number, vendor name, tax ID (GSTIN), bank account and IFSC routing code, PO reference, dates, line items, subtotal, tax and total. Three further fields support later stages: *is_invoice*, *document_type* and a list of *forensic observations*, which records visible tampering and pressure language. The instructions forbid inventing values: missing fields must be returned as null, and amounts must be plain numbers. A post-processing step trims strings and normalises dates to ISO 8601. Any date that fails validation is discarded rather than guessed.

B. Stage 2: Deterministic Rule Engine

Nine checks compare the extracted fields with the database (Table I). Each check returns *pass*, *warn*, *fail* or *skip*, together with a human-readable explanation and a weight w_i. Passing and skipped checks carry zero weight. Identifiers (tax IDs, account numbers, invoice and PO numbers) are compared after removing everything except upper-case alphanumerics. Vendor names are lower-cased and stripped of punctuation and legal suffixes such as “Pvt”, “Ltd” and “LLP”, then compared with the Sørensen–Dice coefficient [21] over character bigrams:

$$D(x, y) = 2|B(x) \cap B(y)| \,/\, (|B(x)| + |B(y)|) \tag{1}$$

where $B(\cdot)$ is the multiset of character bigrams. A vendor is matched first by tax ID, then by name. A Dice score of 1 counts as an exact match. A score of at least 0.86 that is not exact is flagged as a *similar name*, a possible impersonation. The rule risk score is

$$R = \min(100, \textstyle\sum_i w_i) \tag{2}$$

TABLE I
Rule Checks and Their Failure Conditions

CHECK	FAILS / WARNS WHEN	w
Document type	file is not an invoice	80
Approved vendor	blocked (80); unknown and not backed by goods (40); look-alike name (warn 20)	20–80
Tax registration	GSTIN differs from registry	35
Bank details	account (50) or routing code (35) differs	35–50
Duplicate	same file hash (70); same vendor + number (60); same vendor, amount and date (warn 25)	25–70
Arithmetic	qty × rate, line sum, subtotal + tax or total disagree beyond 0.5%; tax > 30%	30
Goods received	billed goods not received, over quantity, tax-ID conflict, already billed	15–55
Purchase order	missing (warn), unknown, other vendor, closed, or over remaining balance	15–40
Dates	future-dated (25); > 1 year old, due before issue, or predates PO (warn 10)	10–25

Checks with weight 60 or more are *hard failures*: a non-invoice, a blocked vendor or an exact duplicate. A PO is considered exceeded when the invoice total is more than 102% of the PO balance left after earlier verified invoices.

C. Goods-Receipt Matching for First-Time Suppliers

When the vendor is not in the registry, the tax-ID and bank checks have nothing to compare against. ProcureMind instead asks a physical question: *did we receive what this invoice bills?* The warehouse records every delivery as a goods received note (GRN), with the supplier name, the tax ID printed on the delivery challan, and the item, quantity and unit cost of each line. Matching an invoice to GRNs proceeds in four steps.

1) Supplier identity: A GRN belongs to the invoicing supplier if its tax ID equals the invoice's tax ID, or if the two names have a Dice score of at least 0.86 and at most one side has a tax ID. A name match with two *different* tax IDs is a *conflict*, the signature of someone impersonating a supplier who really did deliver (weight 45). Only GRNs dated from 180 days before the invoice to 7 days after it are considered.

2) Double-billing protection: A GRN is *claimed* once a verified or pending-review invoice has matched it. Claimed GRNs are excluded from the pool, so the same delivery cannot be billed twice, even under a different invoice number (weight 55 for a new supplier). Received quantities on the remaining open GRNs are pooled per inventory item, with a weighted-average unit cost.

3) Line matching: Invoice descriptions rarely repeat catalogue names verbatim. They add pack sizes, reorder words and abbreviate. For an invoice line d and an inventory item k with name tokens T_k, filler words ("pack", "box", "pcs"…) are removed and plurals stripped, and the score is

$$s(d, k) = 0.5 \cdot |T_k \cap T_d| / |T_k| + 0.5 \cdot D(T_k, T_d) + c(d, k) \qquad (3)$$

where the first term measures how much of the item's name the line covers. If the line quotes the item's SKU, $s = 1$ regardless. The corroboration term $c = 0.25$ applies when the billed quantity equals the open received quantity and the unit price is within 1% of the received cost. It lets an abbreviation such as "MCB 32A SP" match "MCB 32A single pole", but on its own it is too small to produce a match. A line is matched to the best item when $s \geq 0.55$.

4) Line verdicts: Each matched line is *over quantity* if it bills more units than remain received, *price mismatch* if its unit price exceeds the received cost by more than 5%, and otherwise *matched*. Unmatched lines are *not received*. The check fails (weight 50 for a new supplier) on any over-quantity line, or when unmatched lines account for more than 5% of the invoice value. It only warns (weight 15) for price mismatches or minor unmatched lines, such as freight. When every line matches, the invoice is *confirmed by goods received*. The vendor check then becomes a warning (weight 20) that asks the reviewer to register the supplier and confirm their bank details by phone before the first payment, and the PO requirement is waived because the delivery itself is the evidence.

D. Stage 3: Tool-Using Review Agent

The extraction, check results, goods-receipt match and rule score are passed to a second LLM call that has five read-only tools: `search_vendors`, `get_vendor_profile`, `get_vendor_invoice_history`, `get_purchase_order` and `get_goods_receipts`. Following the ReAct pattern [19], the agent alternates between reasoning and tool calls for up to six rounds, and tool use is disabled in the last round so that a verdict is guaranteed. Its instructions tell it to treat rule results as established facts, to weigh known fraud patterns (changed bank details, look-alike names, split billing, round-number totals, amounts far above history, pressure language), and to calibrate its score into the same three bands the decision uses. It returns, as structured output, a verdict (*authentic*, *suspicious* or *fraudulent*), a score $A \in [0, 100]$, a plain-language summary and specific findings ranked by severity. Every tool call is recorded and shown to the reviewer.

E. Stage 4: Decision Fusion

The final risk is $r = \max(R, A)$. An invoice is **rejected** if it has a hard failure or $r \geq 70$. Otherwise it **needs review** if any check failed or $r \geq 30$, and is **verified** in all other cases. A failed check also raises r to at least 30, and a hard failure to at least 70. Taking the maximum gives the key safety property: the LLM can escalate an invoice that the rules missed, for example one with pressure language or an unusual amount, but it cannot argue away a failed check. Reviewers can approve, reject or re-analyse any invoice, and every action is written to an audit trail.

F. Implementation

ProcureMind is a Next.js 16 application written in TypeScript. It uses Drizzle ORM on PostgreSQL, and uploaded files are stored as `bytea`. It is deployed as a single Cloudflare Worker [22] through the OpenNext adapter, with the database on PlanetScale Postgres reached over TLS. Because a Worker cannot reuse a socket opened by another request, the code opens one database client per request. Long analyses are kept alive after the response begins streaming us-

ing the Workers `waitUntil` mechanism. The inventory module gives the warehouse forms to record deliveries and shows which deliveries are still awaiting an invoice.

IV. Experimental Results

A. Experimental Setup

We built a benchmark of ten invoices, each covering a distinct behaviour (Table II): two genuine invoices, one from a registered vendor and one from a first-time supplier whose goods were received, and eight anomalies. The invoices are digitally generated PDFs in an Indian GST layout. Each trial runs against a freshly seeded database containing six registered vendors, six POs, eighteen inventory items, ten GRNs (three of them from unregistered suppliers) and seventeen historical invoices spread over thirty days. The ten invoices are uploaded in order through the production code path: `createInvoiceFromUpload` followed by `analyzeInvoice`, with no mocks. We ran three independent trials, giving 30 analyses. The LLM was OpenAI *gpt-6-luna* with reasoning effort set to *low*. The application ran on a laptop against a local PostgreSQL 16 database, so the measured times include network round trips to the model API.

We treat an anomaly as the positive class. A *true positive* is an anomalous invoice that is not verified; a *false positive* is a genuine invoice that is not verified. *Exact agreement* means the outcome matches the expected one in Table II, with escalation from needs-review to rejected also accepted. To isolate each component we also derive, from the same runs, a *rules-only* decision (the fusion rule with A omitted) and an *agent-only* decision, in which *authentic*, *suspicious* and *fraudulent* map to verified, needs review and rejected.

B. Extraction Accuracy

Six key fields were compared with the text printed on each PDF (read with `pdftotext`): invoice number, GSTIN, PO number, grand total, bank account and the number of line items. All 180 field comparisons (6 fields × 30 analyses) were correct, and so were all ten invoice types. This result is expected for clean, digitally generated PDFs. It should not be read as a measure of accuracy on scanned or photographed invoices.

C. Detection Performance and Ablation

Table III compares the three decision paths. The hybrid system detected every anomaly in all 30 analyses (recall 1.00), and 27 of 30 outcomes agreed exactly with the expected ones. All three errors were false positives on genuine invoices, and each was caused by the agent. In two of the three trials it rated invoice 07, the first-time supplier whose goods were confirmed, as *suspicious* (risk 38), and in one trial it did the same for invoice 01 (risk 35). Its reasons were in line with policy but stricter than the benchmark labels. For invoice 07 it wrote that the supplier's bank account "cannot be checked against registered details" and recommended holding the first payment until the account was confirmed.

Table II
Per-Invoice Outcomes over Three Trials

#	Scenario	Failed checks	R	Exp.	T1 T2 T3	r
01	Genuine, registered	—	0	V	V **F** V	8–35
02	Bank changed, urgent	bank	50	X/F	X X X	82–92
03	Inflated printed total	arithmetic	30	F	F X F	58–72
04	Look-alike vendor	tax, bank, goods	100	X	X X X	100
05	Re-used invoice no.	duplicate	60	X	X X X	78–85
06	Exceeds PO balance	PO	30	F	F F F	48–55
07	New, goods received	—	20	V	**F** V **F**	24–38
08	New, over-billed	vendor, goods	100	X	X X X	100
09	New, nothing received	vendor, goods	100	X	X X X	100
10	New, impersonated	vendor, goods	100	X	X X X	100

V = verified, F = needs review, X = rejected. R = rule score (identical in every trial); r = final risk range over the trials. Bold marks outcomes that differ from the expected one. "New" = first-time supplier.

Table III
Ablation of the Decision Paths (30 Analyses)

Decision path	TP	FP	TN	FN	Prec.	Rec.	F1	Exact
Rules only	24	0	6	0	1.00	1.00	1.00	30/30
Agent only	24	3	3	0	0.89	1.00	0.94	24/30
Hybrid, max(R, A)	24	3	3	0	0.89	1.00	0.94	27/30

On this benchmark the rule engine alone was exact. The agent nevertheless added value in two ways. First, it *escalated the most dangerous attack*, the bank-detail change with pressure language in invoice 02. The rules scored it at 50 (needs review), but the agent raised it to rejection in all three trials, citing the "account frozen" text as a likely account-takeover tactic. Second, it wrote a specific next step for the reviewer in every case, such as contacting the vendor on the registered number and not the one printed on the invoice. Used alone, however, the agent was less accurate: it returned *suspicious* for the duplicate (05) that the rules hard-reject, and it produced the three false positives. Fusion recovered three of the agent's six exact-outcome errors, because the hard-fail rules overrode its weaker verdicts. The agent was also not internally consistent. On invoice 02 it gave the verdict *suspicious* with scores of 82–92, which fall in its own "likely fraud" band.

Goods-receipt matching. Without GRN evidence, invoices 07–10 would all fail in the same way, as invoices from an unknown vendor. With GRN matching, the rules verified 07 in every trial and rejected 08–10, each with a specific reason: 80 helmets billed when 50 were received, safety vests that were never delivered, and a GSTIN that differed from the one recorded at delivery. In all twelve analyses of these four invoices, the agent called `get_goods_receipts` on its own initiative.

Consistency. The rule results were identical in every trial. The agent's verdict was the same in all three trials for 8 of the 10 invoices, and the final outcome for 7 of the 10. All the variation came from borderline cases near the 30-point review threshold (01, 07) or the 70-point rejection threshold (03).

D. Line-Item Matcher Benchmark

To test (3) on its own, we wrote 36 invoice-style descriptions of the 18 catalogue items, two per item: one padded with pack sizes and notes, and one reordered or abbreviated (e.g., "Mesh desk organizer", "MCB 32A SP"). We also wrote 16 descriptions of products not in stock: 8 deliberately close to a catalogue item and 8 unrelated (Table IV). The matcher ranked the correct item first for all 36 positives. At the 0.55 threshold it accepted 33 of them; the three misses were "MCB 32A SP" (0.52), "Assembled PCB, revision C" (0.54) and "Hi-vis vest, reflective" (0.27). When the quantity and price also agree, as they do for a genuine invoice, the corroboration bonus raises acceptance to 35/36. No unrelated product came close to matching (highest score 0.16). The hard negatives exposed a real weakness: 3 of the 8 crossed the threshold, namely "27-inch IPS monitor" matched to the 24-inch monitor, "Safety shoes, ISI marked" to the helmet, and "Copper lugs 2.5 sq mm" to the copper wire. Each such false match is still checked for quantity and price against the delivery. A difference in price, however, only produces a warning, so attribute-aware matching, covering sizes, ratings and product type, is needed.

TABLE IV
Product Matcher on Paraphrased Descriptions

SET	N	METRIC	RESULT
Catalogue items	36	top-1 correct	36 (100%)
		accepted at $s \geq 0.55$	33 (91.7%)
		accepted with corroboration	35 (97.2%)
		mean score	0.82
Unrelated products	8	false matches	0 (max 0.16)
Near-miss products	8	false matches	3 (37.5%)

E. Latency and Cost of Investigation

Table V breaks down the time taken by each stage. An analysis took 12.5 s on average (9.6–16.4 s). Almost all of it was spent on the two LLM stages, while the nine rule checks, including every database query, took 12.5 ms on average, about 0.1% of the total. The agent made 3.0 ± 0.9 tool calls per invoice. It called `search_vendors` 26 times, `get_vendor_profile` and `get_vendor_invoice_history` 18 times each, `get_purchase_order` 15 times and `get_goods_receipts` 14 times. It never needed more than two rounds. Because results are streamed stage by stage, the reviewer sees the extracted fields after about 5 s and the rule results almost immediately after that. On the deployed Worker an end-to-end analysis of invoice 09 produced the same outcome, rejected with risk 100.

TABLE V
Stage Latency over 30 Analyses

STAGE	MEAN	SD	MIN	MAX
1. Extraction (LLM)	5.17 s	1.09 s	3.21 s	8.13 s
2. Rule checks (9, SQL)	12.5 ms	4.7 ms	4 ms	22 ms
3. Agent review (LLM + tools)	7.30 s	1.33 s	5.45 s	10.56 s
4. Decision + persist	6.5 ms	1.8 ms	3 ms	12 ms
Total	**12.50 s**	1.66 s	9.62 s	16.39 s

F. Threats to Validity

The benchmark is small and synthetic, and it was written by the same team that designed the rules. The exact score of the rules-only path should therefore be read as a check that the system is internally consistent, not as evidence that it would generalise. All invoices are clean digital PDFs, only one model was evaluated, and each configuration was run three times. A meaningful estimate of accuracy would require a larger corpus of real, anonymised AP data that includes scanned documents and naturally occurring fraud.

V. CONCLUSION

We presented ProcureMind, an invoice-authentication system. It uses LLMs to read invoices and to investigate anomalies, while the payment decision stays tied to deterministic checks against the organisation's own records. The main methodological contribution is goods-receipt matching. It authenticates a first-time supplier by asking whether the goods it bills were physically received, and it protects against tax-ID impersonation and against billing the same delivery twice. On our benchmark the system caught every anomalous invoice in 30 end-to-end analyses, extracted all key fields correctly and returned a decision in about 12.5 s. Fusing the two scores by taking the stricter of them proved to be a sound safety rule. It let the agent escalate a bank-detail-change attack that the rules alone would only have sent for review, and the hard-fail rules corrected the agent whenever it was too lenient. The experiments also showed the cost of that design: on the benchmark's labels, the agent's caution accounted for every false positive.

Future work falls into four areas. (i) Make the agent consistent: enforce agreement between its verdict and its score, and handle known business conditions such as "first payment to a new supplier" as explicit tasks rather than as added risk. (ii) Use attribute-aware or embedding-based product matching to remove near-miss false matches. (iii) Evaluate on real, scanned and multi-currency invoices. (iv) Add the controls a production system needs, including authentication, separation of duties between the people who record deliveries and those who approve payments, and connection pooling for the serverless database.

ACKNOWLEDGMENT

The authors thank their project guide, Ms. Priyanki Dutta, Presidency University, Bangalore, for her guidance and support throughout this work.

REFERENCES

[1] Association of Certified Fraud Examiners, "Occupational fraud 2024: A report to the nations," ACFE, Austin, TX, USA, 2024.

[2] Federal Bureau of Investigation, "Internet crime report 2023," Internet Crime Complaint Center (IC3), Washington, DC, USA, 2024.

[3] Committee of Sponsoring Organizations of the Treadway Commission, *Internal Control—Integrated Framework*. Durham, NC, USA: AICPA, 2013.

[4] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[5] R. J. Bolton and D. J. Hand, “Statistical fraud detection: A review,” *Statist. Sci.*, vol. 17, no. 3, pp. 235–255, 2002.

[6] E. W. T. Ngai, Y. Hu, Y. H. Wong, Y. Chen, and X. Sun, “The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature,” *Decis. Support Syst.*, vol. 50, no. 3, pp. 559–569, 2011.

[7] J. West and M. Bhattacharya, “Intelligent financial fraud detection: A comprehensive review,” *Comput. Secur.*, vol. 57, pp. 47–66, 2016.

[8] M. J. Nigrini, *Benford's Law: Applications for Forensic Accounting, Auditing, and Fraud Detection*. Hoboken, NJ, USA: Wiley, 2012.

[9] B. Baesens, V. Van Vlasselaer, and W. Verbeke, *Fraud Analytics Using Descriptive, Predictive, and Social Network Techniques*. Hoboken, NJ, USA: Wiley, 2015.

[10] A. R. Katti *et al.*, “Chargrid: Towards understanding 2D documents,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2018, pp. 4459–4469.

[11] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, “LayoutLM: Pre-training of text and layout for document image understanding,” in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 1192–1200.

[12] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, “LayoutLMv3: Pre-training for document AI with unified text and image masking,” in *Proc. 30th ACM Int. Conf. Multimedia*, 2022, pp. 4083–4091.

[13] G. Kim *et al.*, “OCR-free document understanding transformer,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 498–517.

[14] G. Jaume, H. K. Ekenel, and J.-P. Thiran, “FUNSD: A dataset for form understanding in noisy scanned documents,” in *Proc. Int. Conf. Document Anal. Recognit. Workshops (ICDARW)*, 2019, pp. 1–6.

[15] Z. Huang *et al.*, “ICDAR2019 competition on scanned receipt OCR and information extraction,” in *Proc. Int. Conf. Document Anal. Recognit. (ICDAR)*, 2019, pp. 1516–1520.

[16] S. Park *et al.*, “CORD: A consolidated receipt dataset for post-OCR parsing,” in *Proc. Document Intell. Workshop, NeurIPS*, 2019.

[17] OpenAI, “GPT-4 technical report,” arXiv:2303.08774, 2023.

[18] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022, pp. 24824–24837.

[19] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[20] T. Schick *et al.*, “Toolformer: Language models can teach themselves to use tools,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[21] L. R. Dice, “Measures of the amount of ecologic association between species,” *Ecology*, vol. 26, no. 3, pp. 297–302, 1945.

[22] Cloudflare, “Cloudflare Workers documentation.” [Online]. Available: https://developers.cloudflare.com/workers/ (accessed Oct. 24, 2026).